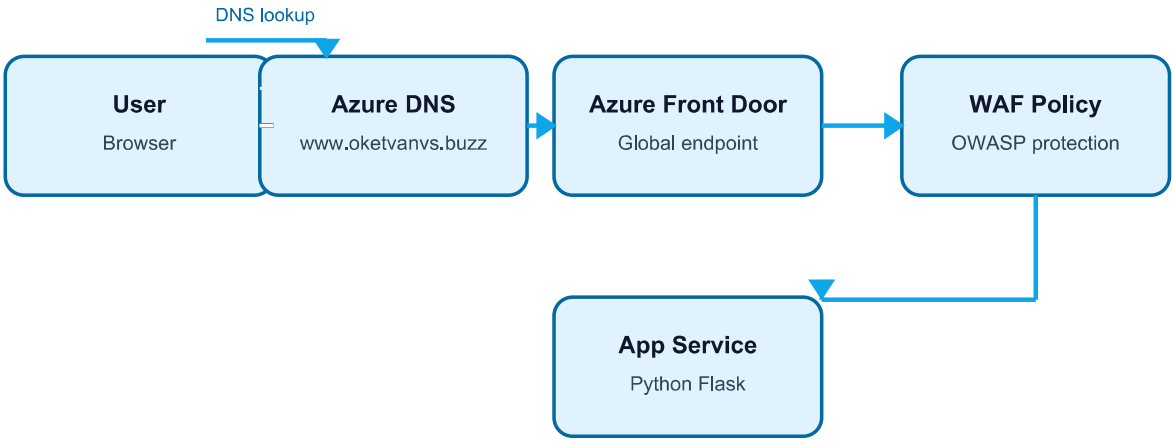
End-to-End Azure Web Application Deployment

Azure App Service + Custom Domain + HTTPS + Azure Front Door + WAF

Professional project documentation for a cloud portfolio and GitHub/LinkedIn showcase.



Final traffic path: User -> Azure DNS -> Front Door -> WAF -> App Service -> Flask app

Final architecture: User -> Azure DNS -> Azure Front Door -> WAF -> Azure App Service -> Flask application

```
Live custom domain: https://www.oketvanvs.buzz
Default App Service domain: oktevanvs-dbdhbeehexbqbmgz.centralindia-01.azurewebsites.net
```

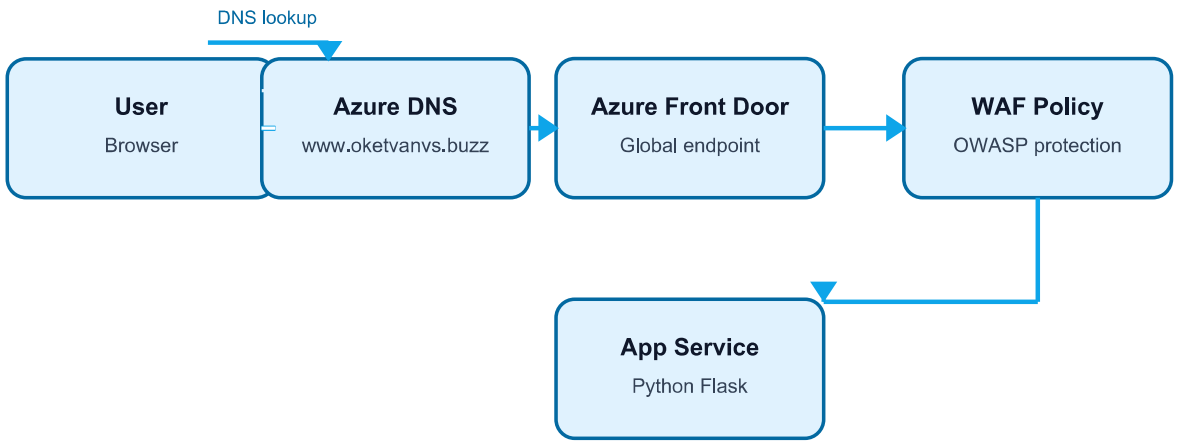
1. Executive Summary

This project demonstrates a secure, production-style web application deployment on Microsoft Azure. A Python Flask application was deployed to Azure App Service, connected to a custom GoDaddy domain through Azure DNS, secured with HTTPS, and then placed behind Azure Front Door with WAF protection.

The implementation focuses on the real-world journey of a web request: browser, DNS, Front Door, WAF, backend routing, App Service runtime, and application response.

Topic	Short summary	Project usage
App Service	Managed PaaS hosting for web applications. Azure manages the underlying server, OS, runtime platform, and routing.	Runs the Python Flask application.
Azure DNS Zone	A DNS record container for your domain. It stores records such as CNAME and TXT.	Manages DNS for oktevanvs.buzz after delegation.
Front Door	Global HTTP/HTTPS entry service for routing, TLS, acceleration, and edge delivery.	Acts as the public entry point before App Service.
WAF	Web Application Firewall that inspects web requests and blocks common attacks.	Attached to Front Door to protect the Flask app.

2. Final Architecture



Final traffic path: User -> Azure DNS -> Front Door -> WAF -> App Service -> Flask app

In the final design, the custom domain points to Azure Front Door. Front Door receives the request, applies WAF inspection, forwards traffic over HTTPS to the App Service origin, and the Flask app returns the response.

```
Final traffic flow:
User -> www.oketvanvs.buzz -> Azure Front Door -> WAF -> App Service origin -> Flask app
```

3. Step-by-Step Implementation

Step 1 - Deploy the Flask application to Azure App Service

A Python Flask application was deployed using Azure App Service and Deployment Center. Deployment Center confirmed that the code deployment was successful. A key learning is that deployment success only means code was published. Runtime success must be verified separately through app logs and browser testing.

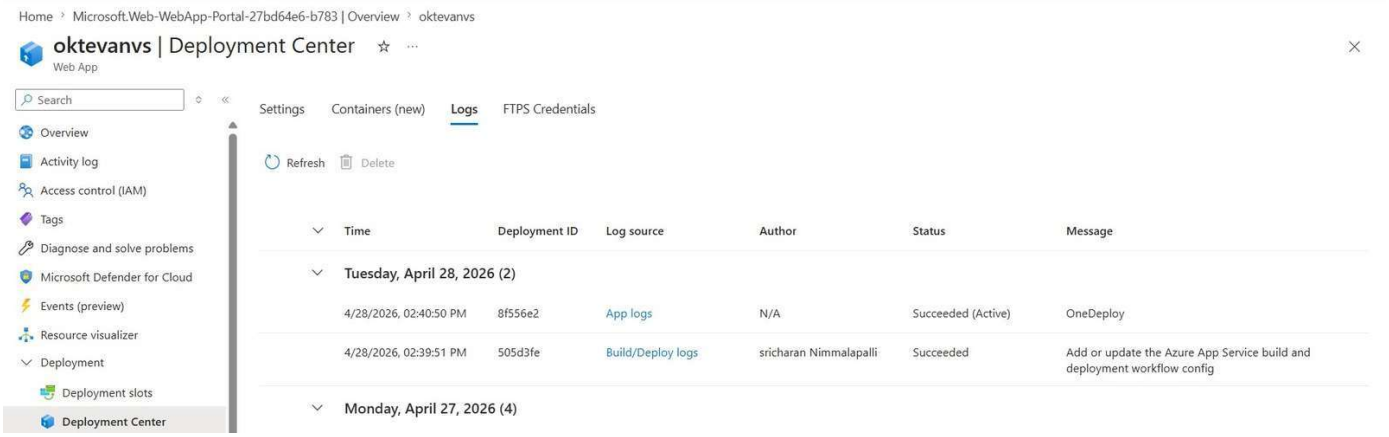


Figure - Deployment Center logs showing successful deployment

```
Correct deployment verification order:
1. Deployment Center status = Succeeded
2. Log stream confirms app startup
3. Restart App Service if needed
4. Test default domain URL
```

Step 2 - Verify the App Service default domain

The default Azure-provided domain was tested first. This confirms the application itself is running before adding DNS, custom domain, Front Door, or WAF layers.

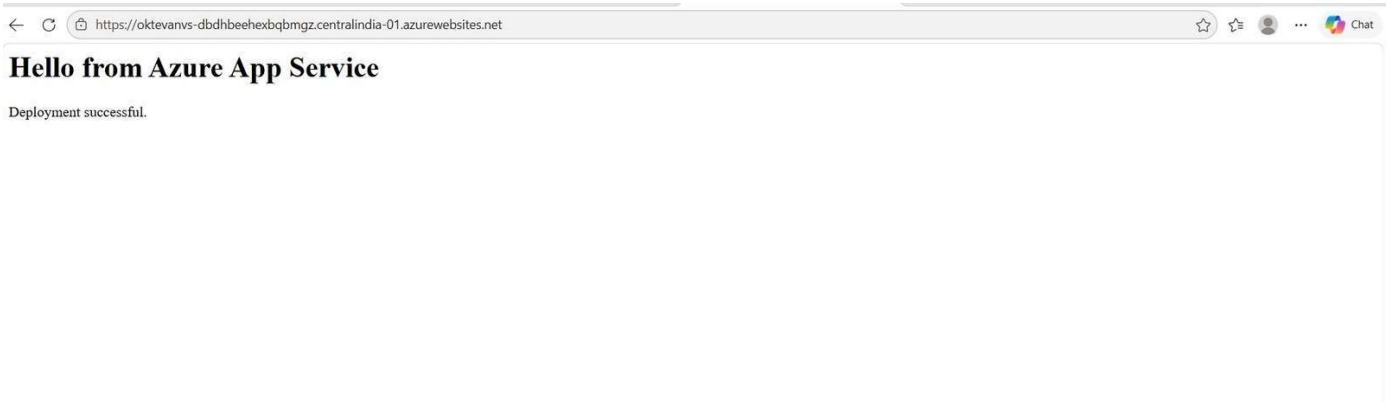


Figure - App Service default domain displaying the Flask app

```
Default App Service URL used for validation:
https://oktevanvs-dbdhbeehexbqbmgz.centralindia-01.azurewebsites.net
```

Step 3 - Create the Azure DNS Zone

A DNS zone was created for the custom domain. A DNS zone is the rule book for a domain. It stores records such as CNAME, A, and TXT. Creating the DNS zone means Azure is ready to manage records, but GoDaddy must delegate DNS authority to Azure by using Azure nameservers.

Figure - Azure DNS Zone created for the domain

Important: If you delete and recreate a DNS zone, Azure generates a new set of nameservers. In that case, GoDaddy must be updated again with the new nameservers.

Step 4 - Understand DNS query and CNAME behavior

DNS resolution was understood as a step-by-step process. The user enters the domain, the browser starts a DNS query, Azure DNS checks the records, the CNAME points to another hostname, that hostname resolves to an IP address, and the browser finally connects to the application.

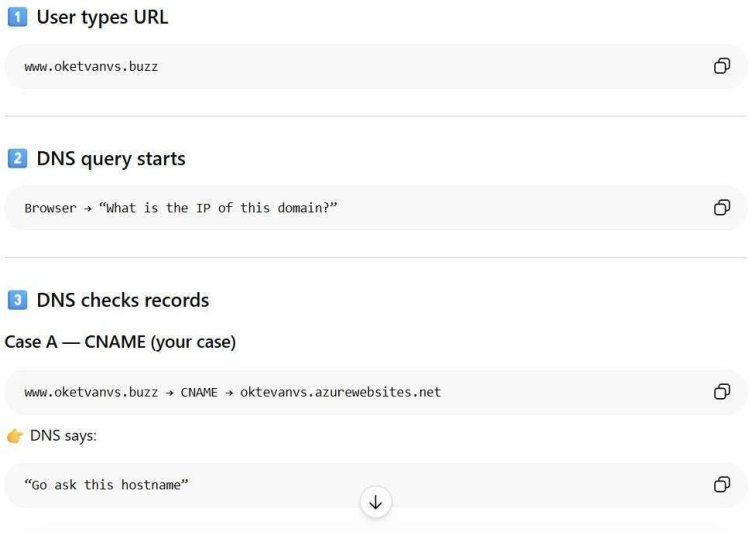


Figure - DNS query flow: user URL, query starts, CNAME lookup

Figure - DNS query flow: target hostname, IP resolution, browser connection

```
DNS query result path:
www.oktevanvs.buzz -> CNAME -> Azure Front Door endpoint -> IP address -> Front Door -> App
Service
```

Step 5 - Add custom domain validation records

When adding the custom domain, Azure required ownership validation. A CNAME record routes traffic, while a TXT record proves domain ownership. Both must be correctly created before Azure can attach the custom domain.

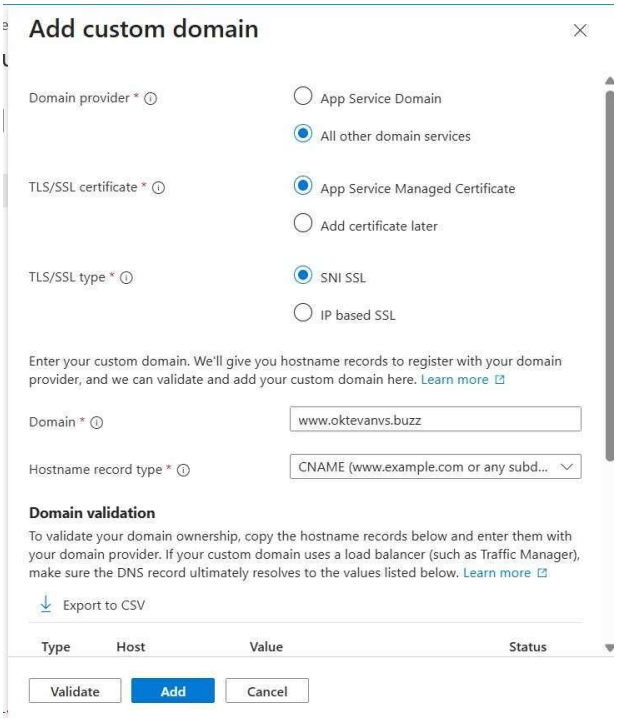


Figure - Custom domain validation using CNAME and TXT records

Topic	Short summary	Project usage
CNAME	Maps one name to another hostname. It does not directly map to an IP.	www.oktevanvs.buzz points to the Azure target hostname.
TXT	Stores text data in DNS. Often used for ownership verification.	asuid TXT record proved that the domain belongs to you.
A record	Maps a domain directly to an IP address.	Not used for App Service in this project because App Service uses a hostname.
TTL	Time to Live. It controls how long DNS resolvers cache the record.	Use low TTL during testing; increase after the setup is stable.

Step 6 - Bind the custom domain and enable HTTPS

The custom domain was attached to the App Service and secured using an App Service Managed Certificate with SNI SSL. Certificate binding means the certificate is attached to the hostname so browsers can establish a trusted HTTPS connection.

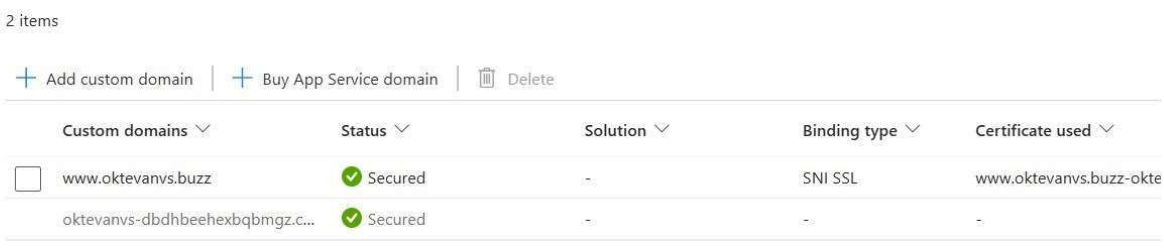


Figure - Custom domain secured with SNI SSL certificate binding

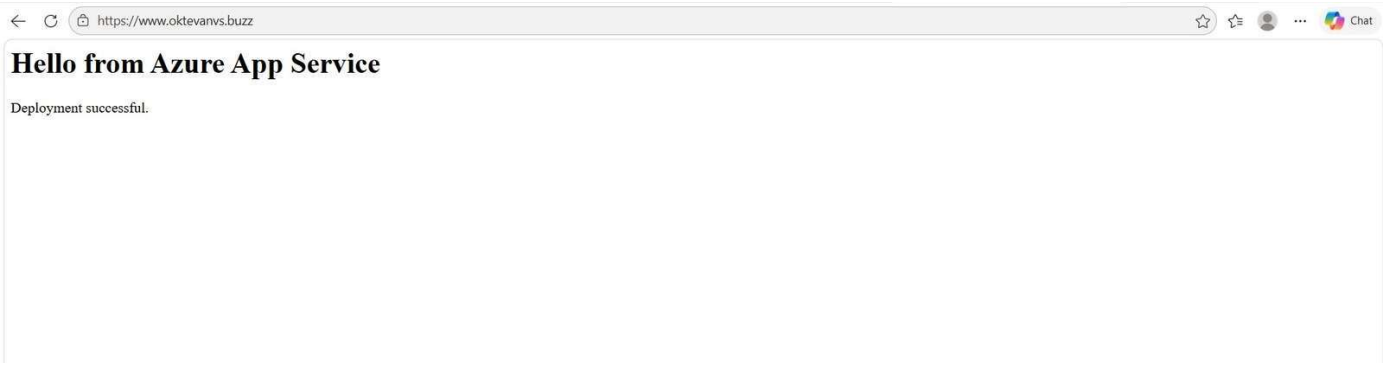


Figure - Custom domain displaying the Flask app over HTTPS

```
Custom domain after HTTPS binding:
https://www.oktevanvs.buzz
```

Key concept: Certificate binding = HTTPS enabled. HTTPS Only = forces users from HTTP to HTTPS.

Step 7 - Create Azure Front Door with WAF

Azure Front Door was created as the global entry point. The App Service was selected as the origin, caching was disabled for this dynamic Flask demo, and a WAF policy was attached. WAF is not automatic; it must be created and associated.

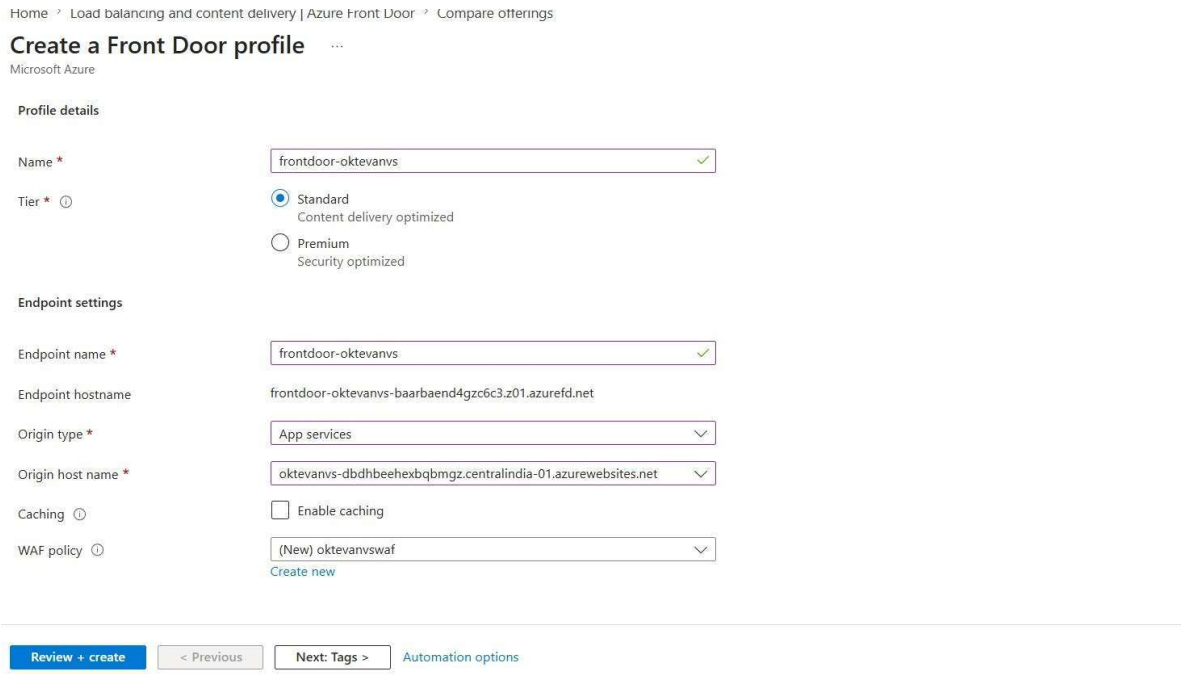


Figure - Front Door profile creation with App Service origin and WAF policy

Topic	Short summary	Project usage
Origin type	The backend type that Front Door forwards traffic to.	App services was selected because the backend is Azure App Service.
Origin hostname	The backend hostname Front Door uses to reach the application.	Set to the App Service default domain.
WAF policy	Security rules that inspect and block malicious HTTP/S traffic.	Attached to Front Door to protect the Flask app.
Caching	Stores responses at edge locations.	Disabled because the Flask page is treated as dynamic content.

Step 8 - Configure the Front Door origin group

The origin group was configured to send traffic to the App Service. The health probe was enabled so Front Door can continuously check whether the backend is healthy.

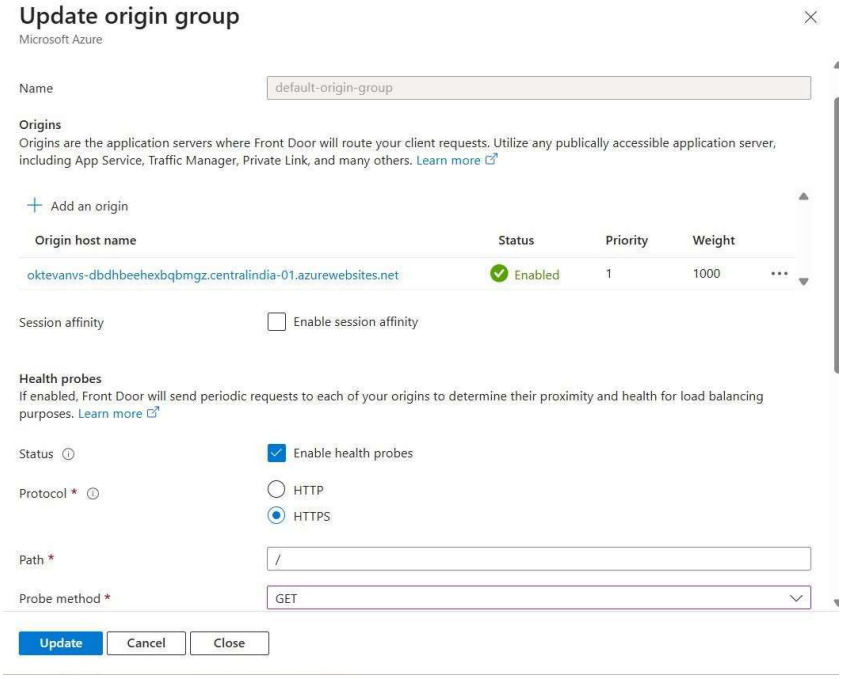


Figure - Front Door origin group connected to App Service

Figure - Front Door health probe configured with HTTPS and GET

```
Correct health probe settings:
Path: /
Protocol: HTTPS
Probe method: GET
Interval: 10 seconds
```

GET was used instead of HEAD because the Flask route clearly responds to browser-style GET requests. This avoids confusing health probe behavior.

Step 9 - Configure the Front Door route

The default route was used with pattern /* so all paths are forwarded to the App Service origin group. The forwarding protocol was set to HTTPS only. This keeps communication from Front Door to App Service secure and consistent.

Figure - Front Door route forwarding protocol set to HTTPS only

```
Correct route settings:
Pattern: /*
Accepted protocols: HTTP and HTTPS
Forwarding protocol: HTTPS only
Caching: Disabled
Route enabled: Yes
```

Step 10 - Update DNS to make the custom domain use Front Door

Creating Front Door does not automatically route the custom domain through it. DNS decides the path. The final required action is to update the CNAME record so www points to the Front Door endpoint instead of directly to App Service.

```
Before Front Door:
www.oktevanvs.buzz -> App Service default domain

After Front Door:
www.oktevanvs.buzz -> Front Door endpoint -> WAF -> App Service
```

Important: Creating Front Door is not the same as using Front Door. Traffic uses Front Door only after DNS points the custom domain to the Front Door endpoint.